

Main [urls.py](#) : need to be update

```
from django.urls import path, include
import User.views as user_views
import rag.views as rag_views
from rest_framework_simplejwt.views import
TokenObtainPairView, TokenRefreshView

urlpatterns = [
    path('admin/', admin.site.urls),
    path('users/', user_views.userApi),
    path('users/<int:pk>/', user_views.userApi),
    path('users/<int:user_id>/chats/',
user_views.chatApi),
    path('users/<int:user_id>/chats/<int:pk>/',
user_views.chatApi),
    path('api/token/',
TokenObtainPairView.as_view(),
name='token_obtain_pair'),
    path('api/token/refresh/',
TokenRefreshView.as_view(), name='token_refresh'),
]
```

Rag views

```
import os
import re
import fitz
import faiss
import pickle
from django.conf import settings
from rest_framework.decorators import api_view
from rest_framework.response import Response
```

```
from sentence_transformers import SentenceTransformer
from google import genai
from User.models import User, Chats
```

```
DATA_PATH=os.path.join(settings.BASE_DIR,"rag","data")
INDEX_PATH=os.path.join(DATA_PATH,"vector_db","legal.index")
CHUNKS_PATH=os.path.join(DATA_PATH,"vector_db","chunks.pkl")
model=SentenceTransformer("all-MiniLM-L6-v2")
```

```
def get_chunks(text):
    chunks=[]
    pattern=r'(?m)^\s*(\d+[A-Z]?)\.\s+(.+)?(?=\n|$)'
    matches=list(re.finditer(pattern,text))
    if not matches:
        paragraphs=re.split(r'\n\s*\n',text)
        current_chunk=""
        for paragraph in paragraphs:
            paragraph=paragraph.strip()
            if not paragraph:
                continue
            if len(current_chunk)+len(paragraph)<=6000:
                current_chunk+="\n\n"+paragraph
            else:
                if current_chunk:
                    chunks.append(current_chunk.strip())
                    current_chunk=paragraph
        if current_chunk:
            chunks.append(current_chunk.strip())
        return chunks
    for i,match in enumerate(matches):
        start=match.start()
        end=matches[i+1].start() if i+1<len(matches) else len(text)
        section=text[start:end].strip()
        if not section:
            continue
        if len(section)<=6000:
            chunks.append(section)
        else:
            parts=re.split(r'(?=\n\s*(\d+))',section)
            current_chunk=""
            for part in parts:
                part=part.strip()
                if not part:
                    continue
                if len(current_chunk)+len(part)<=6000:
                    current_chunk+="\n\n"+part
```

```

        else:
            if current_chunk:
                chunks.append(current_chunk.strip())
            current_chunk=part
        if current_chunk:
            chunks.append(current_chunk.strip())
    return chunks

```

```

def create_embeddings():
    all_chunks=[]
    for file_name in os.listdir(DATA_PATH):
        if not file_name.lower().endswith(".pdf"):
            continue
        pdf_path=os.path.join(DATA_PATH,file_name)
        doc=fitz.open(pdf_path)
        text=""
        for page in doc:
            text+=page.get_text()+"\n"
        doc.close()
        chunks=get_chunks(text)
        for chunk in chunks:
            all_chunks.append({"text":chunk,"source":file_name})
    if not all_chunks:
        return 0
    texts=[chunk["text"] for chunk in all_chunks]
    embeddings=model.encode(texts,normalize_embeddings=True)
    dim=embeddings.shape[1]
    index=faiss.IndexFlatIP(dim)
    index.add(embeddings)
    os.makedirs(os.path.dirname(INDEX_PATH),exist_ok=True)
    faiss.write_index(index,INDEX_PATH)
    with open(CHUNKS_PATH,"wb") as f:
        pickle.dump(all_chunks,f)
    return len(all_chunks)

```

```

def load_index():
    if os.path.exists(INDEX_PATH):
        index=faiss.read_index(INDEX_PATH)
    else:
        index=None
    if os.path.exists(CHUNKS_PATH):
        with open(CHUNKS_PATH,"rb") as f:
            chunks=pickle.load(f)
    else:
        chunks=[]
    return index,chunks

```

```

@api_view(["POST"])
def ask_question(request):
    query=request.data.get("question")
    chat_id=request.data.get("chat_id")
    user_id=request.data.get("user_id")
    conversation_id=request.data.get("conversation_id")
    if not query:
        return Response({"error":"Question is required"},status=400)
    if chat_id is None:
        return Response({"error":"chat_id is required"},status=400)
    if user_id is None:
        return Response({"error":"user_id is required"},status=400)
    if conversation_id is None:
        return Response({"error":"conversation_id is required"},status=400)
    try:
        chat=Chats.objects.get(id=chat_id,user_id=user_id)
    except Chats.DoesNotExist:
        return Response({"error":"Chat not found"},status=404)
    if conversation_id<0 or conversation_id>=len(chat.chats):
        return Response({"error":"Invalid conversation_id"},status=400)
    index,chunks=load_index()
    if index is None or not chunks:
        return Response({"error":"No legal documents have been indexed yet"},status=400)
    query_embedding=model.encode([query],normalize_embeddings=True)
    k=min(5,index.ntotal)
    similarities,indices=index.search(query_embedding,k)
    context=""
    sources=[]
    for i in range(k):
        index_value=indices[0][i]
        if index_value<0:
            continue
        chunk=chunks[index_value]
        context+=chunk["text"]+"\n\n"
        sources.append(chunk["source"])
    prompt=f"""Answer the question using only the provided context.
Context:
{context}
Question:
{query}
If the context does not contain enough information, say so."""
    client=genai.Client(api_key="enter api key")
    response=client.models.generate_content(model="gemini-3.6-flash",contents=prompt)
    chat.chats[conversation_id].append([query,response.text])
    chat.save()
    return Response({"question":query,"answer":response.text,"sources":list(set(sources))})

```

```
Rag urls need to be update
from django.urls import path
from . import views
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView
```

```
urlpatterns = [
    path('rag/', views.ragApi),
    path('rag/<int:pk>/', views.ragApi),
    path('api/token/', TokenObtainPairView.as_view(), name='token_obtain_pair'),
    path('api/token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),
]
```

User views

```
from rest_framework.decorators import api_view
from rest_framework.response import Response
from .models import User, Chats
from .serializers import UserSerializer, ChatsSerializer
```

```
@api_view(['GET', 'POST', 'PUT', 'PATCH', 'DELETE'])
```

```
def userApi(req, pk=None):
```

```
    if req.method == "GET":
```

```
        if pk:
```

```
            user = User.objects.get(id=pk)
```

```
            serializer = UserSerializer(user)
```

```
            return Response(serializer.data)
```

```
        users = User.objects.all()
```

```
        serializer = UserSerializer(users, many=True)
```

```
        return Response(serializer.data)
```

```
    if req.method == "POST":
```

```
        serializer = UserSerializer(data=req.data)
```

```
        if serializer.is_valid():
```

```
            serializer.save()
```

```
            return Response(serializer.data)
```

```
        return Response(serializer.errors)
```

```
    if req.method == "PUT":
```

```
        user = User.objects.get(id=pk)
```

```
        serializer = UserSerializer(user, data=req.data)
```

```
        if serializer.is_valid():
```

```
            serializer.save()
```

```
            return Response(serializer.data)
```

```
        return Response(serializer.errors)
```

```
    if req.method == "PATCH":
```

```
        user = User.objects.get(id=pk)
```

```
        serializer = UserSerializer(user, data=req.data, partial=True)
```

```

        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors)

    if req.method == "DELETE":
        user = User.objects.get(id=pk)
        user.delete()
        return Response({"message": "User deleted successfully"})

@api_view(['GET', 'POST', 'PATCH', 'DELETE'])
def chatApi(req, user_id, pk=None):
    if req.method == "GET":
        if pk:
            chat = Chats.objects.get(id=pk, user_id=user_id)
            serializer = ChatsSerializer(chat)
            return Response(serializer.data)
        chats = Chats.objects.filter(user_id=user_id)
        serializer = ChatsSerializer(chats, many=True)
        return Response(serializer.data)

    if req.method == "POST":
        user = User.objects.get(id=user_id)
        data = req.data.copy()
        data["user"] = user.id
        serializer = ChatsSerializer(data=data)
        if serializer.is_valid():
            serializer.save()
            return Response(serializer.data)
        return Response(serializer.errors)

    if req.method == "PATCH":
        chat = Chats.objects.get(id=pk, user_id=user_id)
        conversation_id = req.data.get("conversation_id")
        query = req.data.get("query")
        answer = req.data.get("answer")
        if conversation_id is None:
            return Response({"error": "conversation_id is required"}, status=400)
        if conversation_id < 0 or conversation_id >= len(chat.chats):
            return Response({"error": "Invalid conversation_id"}, status=400)
        chat.chats[conversation_id].append([query, answer])
        chat.save()
        serializer = ChatsSerializer(chat)
        return Response(serializer.data)

    if req.method == "DELETE":
        chat = Chats.objects.get(id=pk, user_id=user_id)
        chat.delete()

```

```
return Response({"message": "Chat deleted successfully"})
```

User urls need to be update

```
from django.urls import path
```

```
from . import views
```

```
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView
```

```
urlpatterns = [
```

```
    path('users/', views.userApi),
```

```
    path('users/<int:pk>', views.userApi),
```

```
    path('users/<int:user_id>/chats/', views.chatApi),
```

```
    path('users/<int:user_id>/chats/<int:pk>', views.chatApi),
```

```
    path('api/token/', TokenObtainPairView.as_view(), name='token_obtain_pair'),
```

```
    path('api/token/refresh/', TokenRefreshView.as_view(), name='token_refresh'),
```

```
]
```